

從原型設計到正式環境：使用 Vertex AI 訓練自訂模型

來源：<https://codelabs.developers.google.com/vertex-p2p-training?hl=zh-tw>

步驟數：7

目錄

1. [總覽](#)
2. [Vertex AI 簡介](#)
3. [設定環境](#)
4. [將訓練應用程式程式碼容器化](#)
5. [在 Vertex AI 中執行自訂訓練工作](#)
6. [\[選用\] 使用 Vertex AI Python SDK](#)
7. [清除所用資源](#)

步驟 1 · 約 1 分鐘

1. 總覽

在本實驗室中，您將使用 [Vertex AI](#) 執行自訂訓練工作。

本實驗室是「從原型設計到投入正式環境」系列影片的一部分。您將使用[花卉資料集](#)建立圖片分類模型。如要瞭解詳情，請觀看隨附影片：



。

課程內容

內容如下：

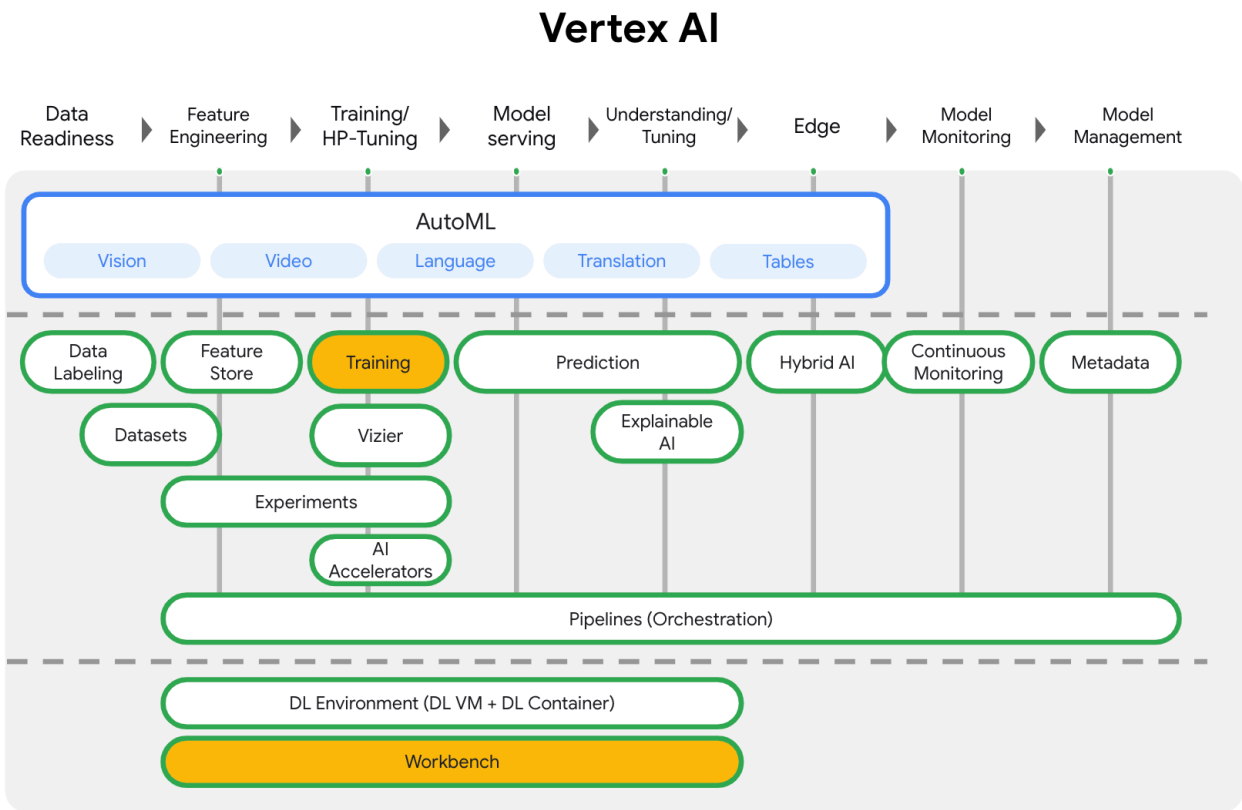
- 建立 Vertex AI Workbench 代管型筆記本
- 透過 Vertex AI 使用者介面設定及啟動自訂訓練工作
- 使用 [Vertex AI Python SDK](#) 設定及啟動自訂訓練工作

在 Google Cloud 上執行這個實驗室的總費用約為 **\$1 美元**。

2. Vertex AI 簡介

本實驗室使用 Google Cloud 最新推出的 AI 產品服務。[Vertex AI](#) 整合了 Google Cloud 機器學習服務，提供流暢的開發體驗。以 AutoML 訓練的模型和自訂模型，先前需透過不同的服務存取。這項新服務將兩者併至單一 API，並加入其他新產品。您也可以將現有專案遷移至 Vertex AI。

Vertex AI 包含許多不同的產品，可支援端對端機器學習工作流程。本實驗室將著重於下列產品：訓練和 **Workbench**



步驟 3 · 約 10 分鐘

3. 設定環境

您必須擁有已啟用計費功能的 Google Cloud Platform 專案，才能執行這項程式碼研究室。如要建立專案，請按照[這裡的說明](#)操作。

步驟 1：啟用 Compute Engine API

前往「Compute Engine」，然後選取「啟用」（如果尚未啟用）。

步驟 2：啟用 Artifact Registry API

前往 [Artifact Registry](#)，然後選取「啟用」（如果尚未啟用）。您將使用這個工具為自訂訓練工作建立容器。

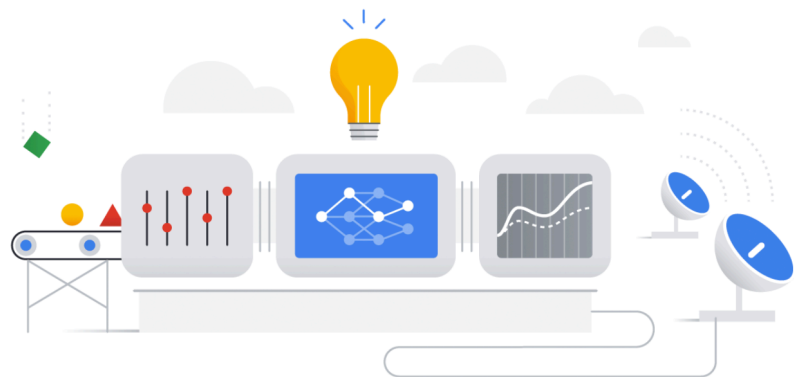
步驟 3：啟用 Vertex AI API

前往 [Cloud 控制台](#) 的 [Vertex AI 專區](#)，然後點選「啟用 Vertex AI API」。

Get started with Vertex AI

Vertex AI empowers machine learning developers, data scientists, and data engineers to take their projects from ideation to deployment, quickly and cost-effectively. [Learn more](#)

ENABLE VERTEX AI API



Region
us-central1 (Iowa)

Prepare your training data

Collect and prepare your data, then import it into a dataset to train a model

+ CREATE DATASET

Train your model

Train a best-in-class machine learning model with your dataset. Use Google's AutoML, or bring your own code.

+ TRAIN NEW MODEL

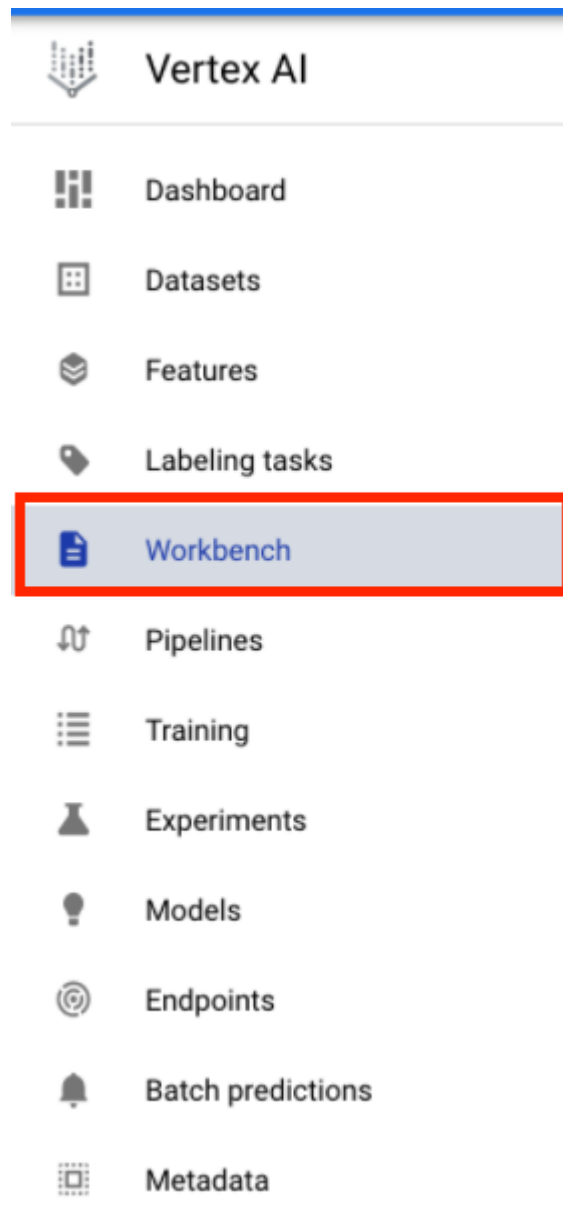
Get predictions

After you train a model, you can use it to get predictions, either online as an endpoint or through batch requests

+ CREATE BATCH PREDICTION

步驟 4：建立 Vertex AI Workbench 執行個體

在 Cloud 控制台的「Vertex AI」部分中，按一下「Workbench」：



如果尚未啟用 Notebooks API，請先啟用。



Notebooks API

Google Enterprise API

Notebooks API is used to manage notebook resources in Google Cloud.

ENABLE

TRY THIS API [↗](#)

OVERVIEW

PRICING

DOCUMENTATION

Overview

Notebooks API is used to manage notebook resources in Google Cloud.

Additional details

Type: [SaaS & APIs](#)

Last updated: 15/09/2021

Category: [Google Enterprise APIs](#)

Service name: notebooks.googleapis.com

啟用後，按一下「MANAGED NOTEBOOKS」(代管型筆記本)：

Managed notebooks

[+ NEW NOTEBOOK](#)

[REFRESH](#)

[▶ START](#)

[■ STOP](#)

[⏻ RESET](#)

[🗑 DELETE](#)

[MANAGED NOTEBOOKS](#) [PREVIEW](#)

USER-MANAGED NOTEBOOKS

EXECUTIONS [PREVIEW](#)

SCHEDULES [PREVIEW](#)

Managed notebooks provide JupyterLab services and flexible computing resources integrated with Google Cloud services. [Learn more](#)

然後選取「新增記事本」。

[+ NEW NOTEBOOK](#)

[REFRESH](#)

[▶ START](#)

[MANAGED NOTEBOOKS](#) [PREVIEW](#)

USER-MANAGED NOTEBOOKS

Notebooks service has been moved under the Vertex AI Workbench

為筆記本命名，然後在「權限」下方選取「服務帳戶」

← Create a managed notebook

Notebook name *

my-notebook

63 character limit with lowercase letters, digits or '-' only. Must start with a letter. Cannot end with a '-'.

Region

us-central1 (Iowa)



Permission

JupyterLab access modes determine who can use a notebook instance and which credentials are used to call Google APIs. **This cannot be changed once the notebook is created.**

☒ Service account

Service account mode allows anyone who is granted the `iam.serviceAccounts.actAs` permission on the specified service account to access the JupyterLab UI. [Learn more](#)

☐ Single user only

Single-user-only mode restricts access to the user specified below. [Learn more](#)

☒ Use Compute Engine default service account

選取「進階設定」。

在「安全性」下方，選取「啟用終端機」(如果尚未啟用)。

Security

☒ Enable nbconvert

☒ Enable file downloading from Notebook UI

☒ Enable terminal

其他進階設定可以保留原樣。

接著點選「建立」。執行個體會在幾分鐘內佈建完畢。

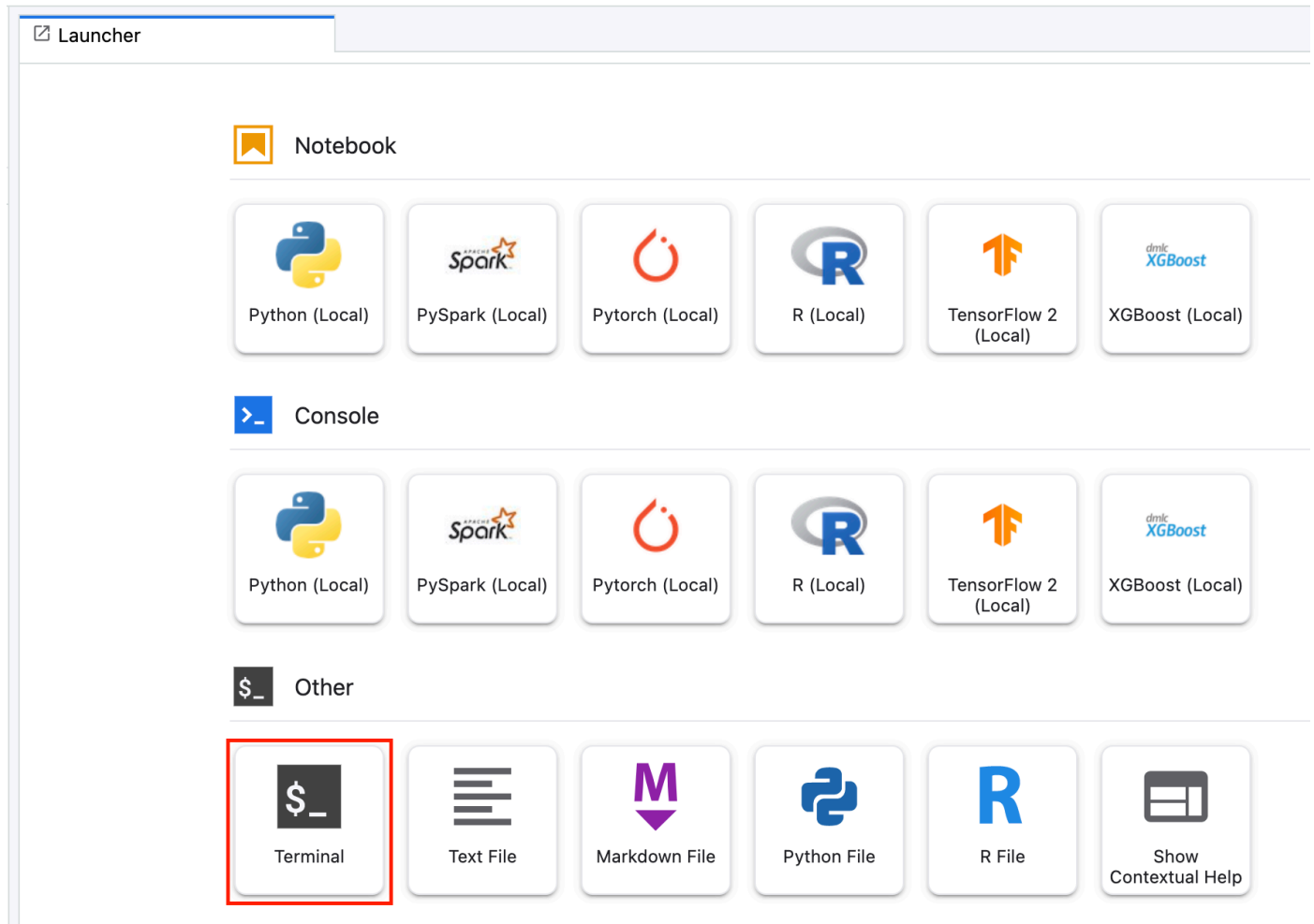
建立執行個體後，請選取「OPEN JUPYTERLAB」。

☐	☒	my-notebook	OPEN JUPYTERLAB
--------------------------	-------------------------------------	-------------	-----------------

4. 將訓練應用程式程式碼容器化

您要將訓練應用程式程式碼放入 [Docker 容器](#)，並將這個容器推送至 [Google Artifact Registry](#)，藉此將訓練工作提交至 Vertex AI。使用這種方法，您可以訓練以任何架構建構的模型。

首先，請從 Launcher 選單開啟筆記本執行個體中的終端機視窗：



步驟 1：建立 Cloud Storage 值區

在這項訓練工作中，您會將經過訓練的 TensorFlow 模型匯出至 Cloud Storage Bucket。您也會將訓練資料儲存在 Cloud Storage 值區中。

在終端機中執行下列指令，為專案定義環境變數，並將 `your-cloud-project` 替換為專案 ID：

您可以在終端機中執行 `gcloud config list --format 'value(core.project)'` 來取得專案 ID

```
PROJECT_ID='your-cloud-project'
```

接著，在終端機中執行下列指令，在專案中建立新的 bucket。

```
BUCKET="gs://${PROJECT_ID}-bucket"  
gsutil mb -l us-central1 $BUCKET
```

步驟 2：將資料複製到 Cloud Storage 值區

我們需要將花朵資料集匯入 Cloud Storage。為方便示範，您會先將資料集下載到這個 Workbench 執行個體，然後複製到 bucket。

下載並解壓縮資料。

```
wget https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz  
tar xvzf flower_photos.tgz
```

然後複製到剛才建立的 bucket。我們新增 -r 是因為要複製整個目錄，新增 -m 則是為了執行多重處理複製作業，加快複製速度。

```
gsutil -m cp -r flower_photos $BUCKET
```

步驟 3：編寫訓練程式碼

建立名為 `flowers` 的新目錄，然後切換至該目錄：

```
mkdir flowers  
cd flowers
```

執行下列指令，為訓練程式碼建立目錄，以及新增程式碼的 Python 檔案。

```
mkdir trainer  
touch trainer/task.py
```

`flowers/` 目錄現在應該包含下列項目：

```
+ trainer/  
  + task.py
```

如要進一步瞭解如何建構訓練應用程式程式碼，請[參閱說明文件](#)。

接著，開啟剛才建立的 `task.py` 檔案，然後複製下列程式碼。

請將 `{your-gcs-bucket}` 改成您剛建立的 Cloud Storage 值區名稱。

透過 [Cloud Storage FUSE 工具](#)，Vertex AI 訓練 的訓練工作可以存取 Cloud Storage 中的資料，就像存取本機檔案系統中的檔案一樣。啟動自訂訓練工作時，工作會看到一個目錄 `/gcs`，其中包含所有 Cloud Storage 值區做為子目錄。因此訓練程式碼中的資料路徑會以 `/gcs` 開頭。

```

import tensorflow as tf
import numpy as np
import os

## Replace {your-gcs-bucket} !!
BUCKET_ROOT='/gcs/{your-gcs-bucket}'

# Define variables
NUM_CLASSES = 5
EPOCHS=10
BATCH_SIZE = 32

IMG_HEIGHT = 180
IMG_WIDTH = 180

DATA_DIR = f'{BUCKET_ROOT}/flower_photos'

def create_datasets(data_dir, batch_size):
    '''Creates train and validation datasets.'''

    train_dataset = tf.keras.utils.image_dataset_from_directory(
        data_dir,
        validation_split=0.2,
        subset="training",
        seed=123,
        image_size=(IMG_HEIGHT, IMG_WIDTH),
        batch_size=batch_size)

    validation_dataset = tf.keras.utils.image_dataset_from_directory(
        data_dir,
        validation_split=0.2,
        subset="validation",
        seed=123,
        image_size=(IMG_HEIGHT, IMG_WIDTH),
        batch_size=batch_size)

    train_dataset = train_dataset.cache().shuffle(1000).prefetch(buffer_size=tf.data.AUTOTUNE)
    validation_dataset = validation_dataset.cache().prefetch(buffer_size=tf.data.AUTOTUNE)

    return train_dataset, validation_dataset

def create_model():
    '''Creates model.'''

    model = tf.keras.Sequential([
        tf.keras.layers.Resizing(IMG_HEIGHT, IMG_WIDTH),
        tf.keras.layers.Rescaling(1./255, input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
        tf.keras.layers.Conv2D(16, 3, padding='same', activation='relu'),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Conv2D(32, 3, padding='same', activation='relu'),
        tf.keras.layers.MaxPooling2D(),

```

```

    tf.keras.layers.Conv2D(64, 3, padding='same', activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(NUM_CLASSES, activation='softmax')
])
return model

# CREATE DATASETS
train_dataset, validation_dataset = create_datasets(DATA_DIR, BATCH_SIZE)

# CREATE/COMPILE MODEL
model = create_model()
model.compile(optimizer=tf.keras.optimizers.Adam(),
              loss=tf.keras.losses.SparseCategoricalCrossentropy(),
              metrics=['accuracy'])

# TRAIN MODEL
history = model.fit(
    train_dataset,
    validation_data=validation_dataset,
    epochs=EPOCHS
)

# SAVE MODEL
model.save(f'{BUCKET_ROOT}/model_output')

```

步驟 4：建立 Dockerfile

如要將程式碼容器化，您需要建立 Dockerfile。Dockerfile 包含執行映像檔所需的所有指令。這會安裝所有必要的程式庫，並設定訓練程式碼的進入點。

在終端機中，於 `flowers` 目錄的根目錄建立空白的 Dockerfile：

```
touch Dockerfile
```

`flowers/` 目錄現在應該包含下列項目：

```

+ Dockerfile
+ trainer/
  + task.py

```

開啟 Dockerfile，然後將下列內容複製到其中：

```
FROM gcr.io/deeplearning-platform-release/tf2-gpu.2-8

WORKDIR /

# Copies the trainer code to the docker image.
COPY trainer /trainer

# Sets up the entry point to invoke the trainer.
ENTRYPOINT ["python", "-m", "trainer.task"]
```

我們來看看這個檔案中的指令。

FROM 指令會指定基本映像檔，也就是您要建立的映像檔所依據的父項映像檔。您將使用 [深度學習容器 TensorFlow 企業版 2.8 GPU Docker 映像檔](#) 做為基本映像檔。Google Cloud 上的深度學習容器預先安裝了許多常見的機器學習和資料科學架構。

WORKDIR 指令會指定後續指令要在映像檔上的哪個目錄執行。

COPY 指令會將訓練員程式碼複製到 Docker 映像檔。請注意，在本範例中，訓練師目錄只有一個 Python 檔案，但若更貼近實際情況，您可能會需要其他檔案。可能一個名為 `data.py` (處理資料預先處理作業)，另一個名為 `model.py` (只包含模型程式碼) 等。如需更複雜的訓練程式碼，請參閱 Python 文件，瞭解如何[封裝 Python 專案](#)。

如要新增其他程式庫，可以使用 **RUN** 指令透過 pip 安裝 (例如 `RUN pip install -r requirements.txt`)。但我們不需要為這個範例新增任何項目。

最後，**ENTRYPOINT** 指令會設定進入點，以叫用訓練工具。這是我們開始訓練工作時執行的內容。在本例中，就是執行 `task.py` 檔案。

如要進一步瞭解如何為 Vertex AI 訓練 編寫 Dockerfile，請參閱[這篇文章](#)。

步驟 4：建構容器

在 Workbench 筆記本的終端機中執行下列指令，為專案定義環境變數，並確保將 `your-cloud-project` 替換為專案 ID：

```
您可以在終端機中執行 gcloud config list --format 'value(core.project)' 來取得專案 ID
```

```
PROJECT_ID='your-cloud-project'
```

在 Artifact Registry 中建立存放區

```
REPO_NAME='flower-app'

gcloud artifacts repositories create $REPO_NAME --repository-format=docker \
--location=us-central1 --description="Docker repository"
```

在 Google Artifact Registry 中，使用容器映像檔的 URI 定義變數：

```
IMAGE_URI=us-central1-docker.pkg.dev/$PROJECT_ID/$REPO_NAME/flower_image:latest
```

設定 Docker

```
gcloud auth configure-docker \
  us-central1-docker.pkg.dev
```

接著，從 `flower` 目錄的根層級執行下列指令，建構容器：

```
docker build ./ -t $IMAGE_URI
```

最後，將其推送至 Artifact Registry：

```
docker push $IMAGE_URI
```

容器已推送至 Artifact Registry，現在可以啟動訓練工作。

步驟 5 · 約 35 分鐘

5. 在 Vertex AI 中執行自訂訓練工作

本實驗室使用 Google Artifact Registry 上的自訂容器進行自訂訓練，但您也可以使用[預先建立的容器](#)執行訓練工作。

首先，請前往 Cloud 控制台的 Vertex 專區，然後點選「訓練」部分：



Vertex AI



Dashboard



Datasets



Features



Labelling tasks



Workbench



Pipelines



Training



Experiments



Models



Endpoints



Batch predictions



Metadata

步驟 1：設定訓練工作

按一下「建立」，輸入訓練工作的參數。

Training

+

CREATE

TRAINING PIPELINES

CUSTOM JOBS

HYPERPARAMETER TUNING JOBS

Training pipelines are the primary model training workflow in Vertex AI. You can use training pipelines to create an AutoML-trained model or a custom-trained model. For

- 在「Dataset」(資料集) 下方，選取「No managed dataset」(沒有代管資料集)
- 然後選取「Custom training (advanced)」(自訂訓練 (進階)) 做為訓練方法，並點按「Continue」(繼續)。
- 選取「訓練新模型」，然後在「模型名稱」中輸入 `flowers-model` (或您想為模型命名的任何名稱)
- 按一下 [Continue] (繼續)。

在「容器設定」步驟中，選取「自訂容器」：

Select a pre-built container or build a custom container using ML frameworks (as well as non-ML dependencies, libraries and binaries) that are not otherwise supported.

[Learn more](#)

☐ Pre-built container

View the list of [supported runtimes](#) including TensorFlow, scikit-learn and PyTorch versions

☒ Custom container

Build a custom Docker container. Must be stored in [Container Registry](#)

在第一個方塊 (「容器映像檔」) 中，輸入上一節的 `IMAGE_URI` 變數值。應為 `us-central1-docker.pkg.dev/{PROJECT_ID}/flower-app/flower_image:latest`，並使用您自己的專案 ID。其餘欄位保留空白，然後點選「繼續」。

再次點選「繼續」，略過「超參數」步驟。

步驟 2：設定運算叢集

請按照下列方式設定「工作站集區 0」：

Worker pool 0

Machine type *

n1-standard-4, 4 vCPUs, 15 GiB memory

Accelerator type

Accelerators can speed up model training that involves intensive compute tasks. [Learn more](#)

Worker count

1

Disk type

SSD

Disk size (GB)

100

✓ ADD MORE WORKER POOLS (OPTIONAL)

CONTINUE

您現在會略過步驟 6，並在本系列課程的下一個實驗室中設定預測容器。

按一下「開始訓練」，啟動訓練工作。在控制台的「訓練」部分，您會看到「訓練管道」分頁下方顯示新啟動的工作：

Training

[+ CREATE](#)

TRAINING PIPELINES

CUSTOM JOBS

HYPERPARAMETER TUNING JOBS

Training pipelines are the primary model training workflow in Vertex AI. You can use training pipelines to create an AutoML-trained model or a custom-trained model. For custom-trained models, training pipelines orchestrate custom training jobs and hyperparameter tuning with additional steps like adding a data set or uploading the model to Vertex AI for prediction serving. [Learn more](#)

Region

us-central1 (Iowa)



Filter Enter a property name

Name	ID	Status	Job type	Model type
flowers-model	7403519214432026624	Training	Training pipeline	Custom

訓練工作大約 10 分鐘即可完成。

恭喜！

您已瞭解如何使用 Vertex AI 執行下列作業：

- 啟動自訂訓練工作，以訓練自訂容器中提供的訓練程式碼。本範例使用 TensorFlow 模型，但您可以使用自訂或內建容器，訓練以任何架構建構的模型。

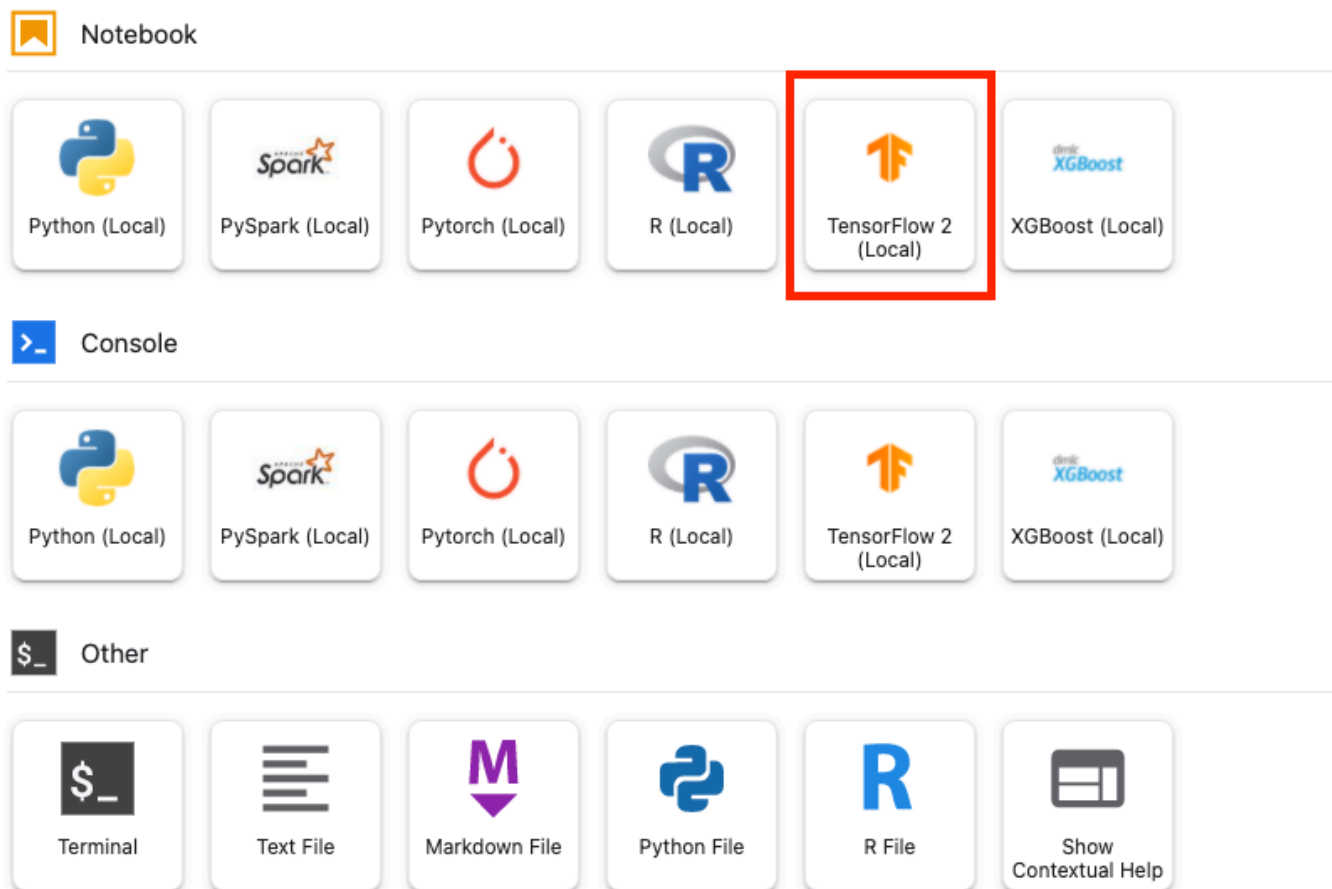
如要進一步瞭解 Vertex 的其他部分，請參閱[說明文件](#)。

步驟 6 · 約 35 分鐘

6. [選用] 使用 Vertex AI Python SDK

上一節說明如何透過 UI 啟動訓練工作。在本節中，您將瞭解如何使用 [Vertex AI Python SDK](#) 提交訓練工作。

返回筆記本執行個體，然後從啟動器建立 TensorFlow 2 筆記本：



匯入 Vertex AI SDK。

```
from google.cloud import aiplatform
```

接著建立 `CustomContainerTrainingJob`。您需要將 `container_uri` 中的 `{PROJECT_ID}` 替換為專案名稱，並將 `staging_bucket` 中的 `{BUCKET}` 替換為您稍早建立的 bucket。

```
my_job = aiplatform.CustomContainerTrainingJob(display_name='flower-sdk-job',
                                              container_uri='us-central1-
docker.pkg.dev/{PROJECT_ID}/flower-app/flower_image:latest',
                                              staging_bucket='gs://{BUCKET}')
```

然後執行工作。

```
my_job.run(replica_count=1,  
           machine_type='n1-standard-8',  
           accelerator_type='NVIDIA_TESLA_V100',  
           accelerator_count=1)
```

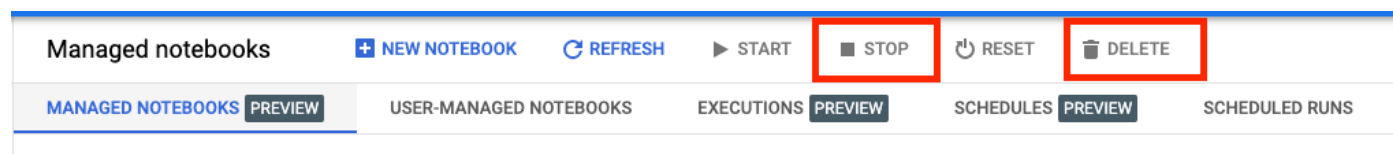
為方便示範，這項工作已設定為在比上一節更大的機器上執行。此外，我們也使用 GPU 執行。如未指定 `machine-type`、`accelerator_type` 或 `accelerator_count`，工作預設會在 `n1-standard-4` 上執行。

在控制台的「訓練」部分，您會看到「自訂工作」分頁下方的訓練工作。

步驟 7 · 約 1 分鐘

7. 清除所用資源

由於 Vertex AI Workbench 代管筆記本具有閒置關機功能，因此我們不必擔心執行個體關機問題。如要手動關閉執行個體，請按一下控制台 Vertex AI Workbench 專區的「停止」按鈕。如要徹底刪除筆記本，請按一下「刪除」按鈕。



如要刪除 Storage Bucket，請使用 Cloud 控制台中的導覽選單瀏覽至 Storage，選取 bucket，然後按一下「Delete」：

